TP2 TCH017

Chiffrement de César, codage de l'information

Charles Pillet

Chargé de cours à l'ÉTS

cc-charles.pillet@etsmtl.ca

Hiver 2026

Prénoms et noms :

.....

.....

.....

1 Chiffrement de Cesar

LSOXFOXEONKXCVONOEHSOWODZNOMOMYEBC

Cette phrase a été préalablement chiffré par un chiffrement par décalage ou dit de César. Le chiffrement est une technique pour cacher le réel sens de votre message à des personnes essayant d'espionner vos communications. Votre message est protégé contre l'espion et déchiffrable par le destinataire car ce dernier possède la clé de déchiffrage alors que l'espion ne l'a pas.

En ayant la clé, le destinataire sait que la phrase était :

BIENVENUEDANSLEDEUXIEMETPDECECOURS

Le chiffrement de César est un algorithme de chiffrement qui se base sur un décalage circulaire des lettres de l'alphabet. Le décalage est fixe pour chaque lettre.

1.1 Description mathématique

1.1.1 L'alphabet de notre message

Un alphabet peut être défini comme un ensemble de symboles, composant notre message. L'alphabet $\mathcal{A}$ utilisé est le suivant.

L'alphabet de notre message

Pour rester simple, nous décidons décrire notre message en lettres majuscules seulement (comme l'exemple), ainsi nous avons l'alphabet

$$\mathcal{A} = \{A, \ldots, Z\}.$$

On note que cet ensemble est de taille $|\mathcal{A}| = 26$. On associe chaque élément de l'alphabet à un élément de l'anneau mathématique $\mathbb{Z}_{26} = \mathbb{Z}/26 = \{0, 1, \ldots, 25\}$ via l'application :

$$\begin{aligned} \varphi : \mathcal{A} &\longrightarrow \mathbb{Z}_{26} \\ A &\mapsto 0 \\ B &\mapsto 1 \\ &\vdots \\ Z &\mapsto 25 \end{aligned}$$

Autrement dit, on associe chaque lettre majuscule à sa position dans l'alphabet (en commençant à 0), i.e.,

$\varphi(A)=0$ $\varphi(B)=1$ $\varphi(C)=2$ $\varphi(D)=3$ $\varphi(E)=4$ $\varphi(F)=5$ $\varphi(G)=6$ $\varphi(H)=7$
$\varphi(I)=8$ $\varphi(J)=9$ $\varphi(K)=10$ $\varphi(L)=11$ $\varphi(M)=12$ $\varphi(N)=13$ $\varphi(O)=14$
$\varphi(P)=15$ $\varphi(Q)=16$ $\varphi(R)=17$ $\varphi(S)=18$ $\varphi(T)=19$ $\varphi(U)=20$ $\varphi(V)=21$
$\varphi(W)=22$ $\varphi(X)=23$ $\varphi(Y)=24$ $\varphi(Z)=25$

L'opération inverse aussi possible, i.e.,

$$\begin{aligned}\varphi^{-1} : \mathbb{Z}_{26} &\longrightarrow \mathcal{A}\\ 0 &\mapsto A\\ 1 &\mapsto B\\ &\vdots\\ 25 &\mapsto Z\end{aligned}$$

Autrement dit, on associe chaque position (commence à 0) à sa lettre dans l'alphabet, i.e.,

$\varphi^{-1}(0)=A$ $\varphi^{-1}(1)=B$ $\varphi^{-1}(2)=C$ $\varphi^{-1}(3)=D$ $\varphi^{-1}(4)=E$ $\varphi^{-1}(5)=F$ $\varphi^{-1}(6)=G$
$\varphi^{-1}(7)=H$ $\varphi^{-1}(8)=I$ $\varphi^{-1}(9)=J$ $\varphi^{-1}(10)=K$ $\varphi^{-1}(11)=L$ $\varphi^{-1}(12)=M$
$\varphi^{-1}(13)=N$ $\varphi^{-1}(14)=O$ $\varphi^{-1}(15)=P$ $\varphi^{-1}(16)=Q$ $\varphi^{-1}(17)=R$ $\varphi^{-1}(18)=S$
$\varphi^{-1}(19)=T$ $\varphi^{-1}(20)=U$ $\varphi^{-1}(21)=V$ $\varphi^{-1}(22)=W$ $\varphi^{-1}(23)=X$ $\varphi^{-1}(24)=Y$
$\varphi^{-1}(25)=Z$

1.1.2 Chiffrement à l'émission

Le décalage de César à l'envoi se réalise de cette manière

$$\varphi(C_i) = \varphi(M_i) + C \quad [26] \tag{1}$$

où C_i, M_i représente la i-ème lettre du code et du message, C est une clé, une valeur entière entre 0 et $|\mathcal{A}| = 26$. Le [26] représente l'opération modulo, c'est à dire le reste de la division par 26.

Dans l'exemple précédent, nous avons la clé $C = 10$, lors de l'envoi du message

— $M_1 = B$, $\varphi(B) + C \quad [26] = 1 + 10 \quad [26] = 11$ ce qui donne la lettre L= $\varphi^{-1}(11)$

  Mais aussi...

— $M_5 = V$, $\varphi(V) + C \quad [26] = 21 + 10 \quad [26] = 5$ ce qui donne la lettre F= $\varphi^{-1}(5)$

  etc...

Question (2 points) Expliquer pourquoi la clé $C = 0$ ou la clé $C = 26$ est inutile.

1.1.3 Déchiffrement à la réception

Le décalage de César à la réception se réalise de cette manière

$$\varphi(M_i) = \varphi(C_i) - C \quad [26] \tag{2}$$

Toujours avec l'exemple précédent, nous avons

— $C_1 = L$, $\varphi(L) - C \quad [26] = 11 - 10 \quad [26] = 1$ ce qui donne la lettre B= $\varphi^{-1}(1)$

Mais aussi...

— $C_5 = F$, $\varphi(F) + C \quad [26] = 5 - 10 \quad [26] = -5 \quad [26] = 21 \quad [26]$ ce qui donne la lettre V= $\varphi^{-1}(21)$

etc...

1.2 Passage du message en ASCII et en codage Cesar en PEP/8

En PEP/8, notre message, définie avec la directive ASCII est encodé en ASCII. Cet encodage permet par exemple l'affichage avec l'instruction `STRO` . D'après la table ASCII affichée en Figure 1, notre alphabet se situe entre le numéro ASCII 65 et le numéro ASCII 90.

Remarque Ainsi, une lettre majuscule chargée dans l'accumulateur aura une valeur entre 65 et 90.

Dans votre programme, vous allez devoir *switcher* entre les valeurs Cesar (0 à 25) et ASCII (65 à 90) Nous définissons deux fonctions, une allant de l'encodage César à l'encodage ASCII :

$$\begin{aligned} \varphi_A : \mathbb{Z}_{26} &\longrightarrow \{65, \ldots, 90\} \\ p &\longmapsto \varphi_A(p) = p + 65 \end{aligned}$$

Et une autre allant de l'encodage ASCII à l'encodage César :

$$\begin{aligned} \varphi_A^{-1} : \{65, \ldots, 90\} &\longrightarrow \mathbb{Z}_{26} \\ a &\longmapsto \varphi_A^{-1}(a) = a - 65 \end{aligned}$$

1.3 Travail à réaliser en PEP/8

Un squelette de code vous est fourni avec différentes directives.

Bloc de code 1 – Les directives présentes par défaut

```
; Divers messages
msgLong: .ASCII  "Le message est de longueur : \x00"
welcome: .ASCII  "Bienvenue dans le TP2\nLe message est : \x00"
; Le message et le futur code (doivent être identique)
msg:     .ASCII  "BienvenuedansledeuxiemeTPdececours\x00"
enc:    .ASCII  "BienvenuedansledeuxiemeTPdececours\x00"
; Taille du message
msgSize: .WORD   0
; Tableau étape intermédiaire : assez d'espace pour faire vos calculs.
msgPhi:   .BLOCK 300 ; Valeur de phi avec message initial
encPhi:   .BLOCK  300 ; Valeur de phi après encodage
decPhi:   .BLOCK  300 ; Valeur de phi après un décodage
```

ASCII tableau

Déc	Hex	Oct	Binaire	Caractère	Déc	Hex	Oct	Binaire	Caractère	Déc	Hex	Oct	Binaire	Caractère	Déc	Hex	Oct	Binaire	Caractère
0	0	0	0	NUL	32	20	40	100000	space	64	40	100	1000000	@	96	60	140	1100000	`
1	1	1	1	SOH	33	21	41	100001	!	65	41	101	1000001	A	97	61	141	1100001	a
2	2	2	10	STX	34	22	42	100010	"	66	42	102	1000010	B	98	62	142	1100010	b
3	3	3	11	ETX	35	23	43	100011	#	67	43	103	1000011	C	99	63	143	1100011	c
4	4	4	100	EOT	36	24	44	100100	$	68	44	104	1000100	D	100	64	144	1100100	d
5	5	5	101	ENQ	37	25	45	100101	%	69	45	105	1000101	E	101	65	145	1100101	e
6	6	6	110	ACK	38	26	46	100110	&	70	46	106	1000110	F	102	66	146	1100110	f
7	7	7	111	BEL	39	27	47	100111	'	71	47	107	1000111	G	103	67	147	1100111	g
8	8	10	1000	BS	40	28	50	101000	(	72	48	110	1001000	H	104	68	150	1101000	h
9	9	11	1001	HT	41	29	51	101001	)	73	49	111	1001001	I	105	69	151	1101001	i
10	0A	12	1010	LF	42	2A	52	101010	*	74	4A	112	1001010	J	106	6A	152	1101010	j
11	0B	13	1011	VT	43	2B	53	101011	+	75	4B	113	1001011	K	107	6B	153	1101011	k
12	0C	14	1100	FF	44	2C	54	101100	,	76	4C	114	1001100	L	108	6C	154	1101100	l
13	0D	15	1101	CR	45	2D	55	101101	-	77	4D	115	1001101	M	109	6D	155	1101101	m
14	0E	16	1110	SO	46	2E	56	101110	.	78	4E	116	1001110	N	110	6E	156	1101110	n
15	0F	17	1111	SI	47	2F	57	101111	/	79	4F	117	1001111	O	111	6F	157	1101111	o
16	10	20	10000	DLE	48	30	60	110000	0	80	50	120	1010000	P	112	70	160	1110000	p
17	11	21	10001	DC1	49	31	61	110001	1	81	51	121	1010001	Q	113	71	161	1110001	q
18	12	22	10010	DC2	50	32	62	110010	2	82	52	122	1010010	R	114	72	162	1110010	r
19	13	23	10011	DC3	51	33	63	110011	3	83	53	123	1010011	S	115	73	163	1110011	s
20	14	24	10100	DC4	52	34	64	110100	4	84	54	124	1010100	T	116	74	164	1110100	t
21	15	25	10101	NAK	53	35	65	110101	5	85	55	125	1010101	U	117	75	165	1110101	u
22	16	26	10110	SYN	54	36	66	110110	6	86	56	126	1010110	V	118	76	166	1110110	v
23	17	27	10111	ETB	55	37	67	110111	7	87	57	127	1010111	W	119	77	167	1110111	w
24	18	30	11000	CAN	56	38	70	111000	8	88	58	130	1011000	X	120	78	170	1111000	x
25	19	31	11001	EM	57	39	71	111001	9	89	59	131	1011001	Y	121	79	171	1111001	y
26	1A	32	11010	SUB	58	3A	72	111010	:	90	5A	132	1011010	Z	122	7A	172	1111010	z
27	1B	33	11011	ESC	59	3B	73	111011	;	91	5B	133	1011011	[	123	7B	173	1111011	{
28	1C	34	11100	FS	60	3C	74	111100	<	92	5C	134	1011100	\	124	7C	174	1111100	|
29	1D	35	11101	GS	61	3D	75	111101	=	93	5D	135	1011101	]	125	7D	175	1111101	}
30	1E	36	11110	RS	62	3E	76	111110	>	94	5E	136	1011110	^	126	7E	176	1111110	~
31	1F	37	11111	US	63	3F	77	111111	?	95	5F	137	1011111	_	127	7F	177	1111111	DEL

GEOCACHEN.BE

FIGURE 1 – La table ASCII.

```
13 ; Lier au ASCII et l'alphabet
14 A:        .WORD 'A' ; 65
15 a:        .WORD 'a' ; 97
16 Z:        .WORD 'Z' ; 90
17 z:        .WORD 'z' ; 122
18 min2maj: .EQUATE 32 ; On doit enlever 32 pour passer d'une minuscule à une
     majuscule.
19
20 sizeA:   .EQUATE 26 ; Taille de l'alphabet
21 ; Décodage
22 CLE:      .EQUATE   10                ; Decalage Cesar
23 ; Analyse fréquentielle
24 msgFreq:          .BLOCK  26
25 codeFreq:          .BLOCK  26
```

1.3.1 Quelques petites remarques avant de commencer

Dans la suite, une série de fonctions sera à réaliser, dans ce TP, nous souhaitons que vous utilisiez la pile.

Remarque Votre code doit faire l'usage de la pile lorsque c'est mentionné, la note de 0 sera attribuée sur la question si vous n'utilisez pas la pile.

Remarque Certaines variables sont considérées comme globales, et peuvent être utilisé directement dans vos sous-programmes.
Exemples : `msg_size`.

Remarque Les squelettes qui suivent mentionnent le retour `RET0`, vous avez le droit de créer des variables locales dans les sous-programmes, si bien que l'instruction de retour ne sera pas `RET0`.

1.3.2 La longueur du message...

Comme indiqué, la directive `msg` et `code` doivent être identiques. Le squelette vous donne une manière de calculer la longueur du code :

Bloc de code 2 – Le calcul de la longueur de `msg`.

```
LDA     enc,i
SUBA    msg,i
STA     msgSize,d
```

Question (2 points) Expliquer pourquoi ce code donne la longueur du message `msg`.

1.3.3 Vérification que le message est valide (4 points)

La première fonction à réaliser est la méthode `verif`, cette méthode doit prendre une référence au `msg` (ainsi son adresse est dans la pile), et vérifier si `msg` a bien tout ses symboles dans $\mathcal{A}$.

Bloc de code 3 – Le sous-programme `verif`.

```
;==========================================================================
; Sous-programe : verif
; Doit vérifier si le message fourni en référence est bien composé de lettre
    majuscule seulement.
; Entree  : Adresse de msg (dans la pile)
; Sortie  : Stopper le programme si mauvais message
;==========================================================================

verif:   RET0
```

1.3.4 Passage du ASCII au non codé (4 points)

La deuxième fonction à réaliser est le sous-programme `phiInvA`.

Bloc de code 4 – Le sous-programme `phiInvA`.

```
;==========================================================================
; Sous-programme : phiInvA
; Applique la fonction phi_Inv_A sur msg, et écris dans vers msgPhi
; Entree  : Adresse du msg (SP+2)
; Sortie  : msgPhi est modifié
;==========================================================================

phiInvA: RET0
```

Les variables `msgPhi` et `msg_size` sont considérés comme globales et peuvent être utilisés directement. La variable `msg`, via son adresse, est transmise via la pile d'appel.

1.3.5 Le codage de César `CESARENC` (8 points)

La troisième fonction à réaliser est le codage de César, en suivant l'équation (1). Les variables `msgPhi`, `msg_size` et `encPhi` sont considérés comme globales et peuvent être utilisés directement.

Bloc de code 5 – Le sous-programme `CESARENC`.

```
;==========================================================================
; Sous-programme : CESARENC
; Effectue le chiffrement de Cesar (msgPhi vers encPhi)
; Entree  : Utilise msgPhi et CLE
; Sortie  : encPhi est updaté
;==========================================================================

CESARENC:RET0
```

1.3.6 L'affichage d'un message dans $\mathbb{Z}_{26}$ (8 points)

La prochaine fonction à implémenter est la fonction `STRO_CES` qui permet d'afficher dans le terminal un tableau à valeur dans $[0, 25]$. Pour afficher ce tableau dans le domaine ASCII, pour chaque élément p du tableau, il faut appliquer $\varphi_A(p) = p + 65$ puis afficher cet élément avec `CHARO`.

Remarque `STRO_CES` doit pouvoir fonctionner avec `msgPhi`, `decPhi` ou `encPhi`.

En se basant sur la remarque précédente, la pile possède l'adresse du début du tableau, `msg_size` est considéré comme globale.

Remarque Il est demandé aux étudiants d'utiliser une variable locale au sous programme pour afficher chaque élément.

Bloc de code 6 – Le sous-programme `STRO_CES`.

```
;==========================================================
; Sous-programme : STRO_CES
; Affiche le message (en format Cesar de 0 à 25) et passe à la ligne
; Entree  : msg (SP+2)
; Sortie  :
;==========================================================

STRO_CES:RET0
```

1.3.7 Le décodage du code `CESARDEC` (8 points)

La dernière fonction de cette partie est le décodage de César, se basant sur l'équation (2). Dans cette fonction, la variable `encPhi` doit être décodé, le résultat étant stocké dans `decPhi`.

Bloc de code 7 – Le sous-programme `CESARDEC`.

```
;==========================================================================
```

```
; Sous-programme : CESARDEC
; Effectue le chiffrement de Cesar (encPhi vers decPhi)
; Entree  : Utilise encPhi et CLE
; Sortie  : decPhi est updaté
;==========================================================================

CESARDEC:RET0
```

Remarque Vous devez afficher `decPhi` avec votre fonction `STRO_CES`.

2 Casser le code Cesar

Dans la première partie, il faisait mention qu'une communication chiffrée peut servir à sécuriser la communication contre des personnes qui écoutent la conversation. Dans la suite, nous allons casser le code de César de deux manières.

2.1 Attaque par force brute (8 points)

L'attaque par force brute (*brute force*) est une méthode utilisée en cryptanalyse pour trouver un mot de passe ou une clé. Il s'agit de tester, une à une, toutes les combinaisons possibles. Dans notre cas, nous avons 25 clés à tester (la clé 0 étant inutile).

Remarque Vous devez afficher dans le programme, la clé testée et le message décodé à chaque étape de la boucle.

Bloc de code 8 – Le sous-programme `FORCE`.

```
;==========================================================
; Sous-programme : FORCE
; Effectue une attaque par force brute, essaie et affiche les 25 clés
; Entree  : decPhi
; Sortie  : Sortie dans le terminal
;==========================================================

FORCE:    RET0
```

2.2 Analyse fréquentiel

Depuis le début du TP, les messages sont analysés "dans le domaine temporel", en effet, on regarde quelle est la première lettre... puis la deuxième puis la troisième. L'analyse temporel nous donne une information sur le message, et nous a permis de décoder le message par force brute. Néanmoins, le message peut dire d'autres informations **grâce à l'analyse fréquentiel.** L'analyse fréquentiel est omniprésent en génie, en télécommunications, par exemple, le signal est transporté sur une fréquence.

Faisons un petit jeu... Pourriez vous deviner de quoi est formé ce signal sinusoïdal bruité ?

Remarque On peut considérer un signal sinusoïdal bruité comme une somme de signal sinusoïdal de fréquences différentes avec l'ajout d'un bruit gaussien.

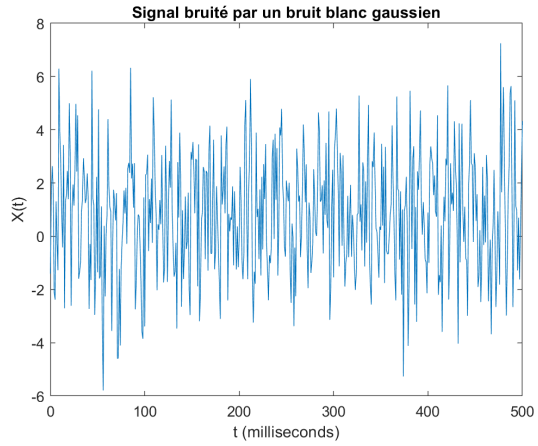

FIGURE 2 – Le signal sinusoïdal

Bien sûr ici, on a le point de vue temporel...

Maintenant, on prend la transformée de Fourier, la transformée de Fourier fait passé notre signal dans le domaine temporel au domaine spectral (les fréquences).

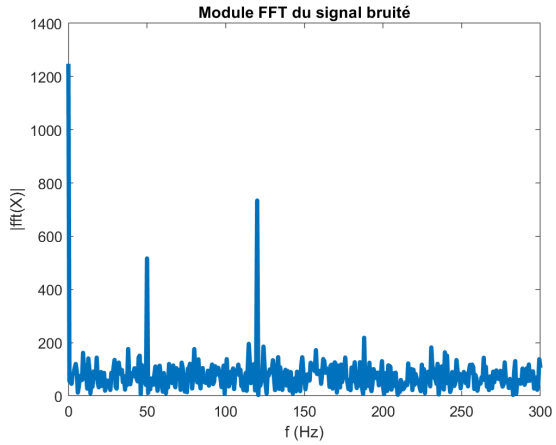

FIGURE 3 – La transformée de Fourier du signal sinusoïdal

Ici, dans le domaine spectral, on voit 3 pics, à 0Hz, à 50 Hz, et à 120Hz, ainsi notre signal possède 3 composantes, imperceptible dans le domaine temporel.

Remarque Si vous continuez en génie électrique, vous verrez les transformées de Fourrier.

2.2.1 Analyse fréquentielle du code de César

Le code de César peut être facilement cassé par analyse fréquentiel, en effet, nous faisons seulement un décalage pour obtenir notre mot encodé. Ainsi, si nous regardons la distribution d'apparition des lettres entre le mot codé et le message initial, la distribution est identique. Exemple : LSOXFOXEONKXCVONOEHSOWODZNOMOMYEBC est le message codé de BIENVENUEDANSLEDEUXIEMETPDECECOURS.

La distribution d'apparition des lettres pour le mot codé et le message est affiché à travers un histogramme.

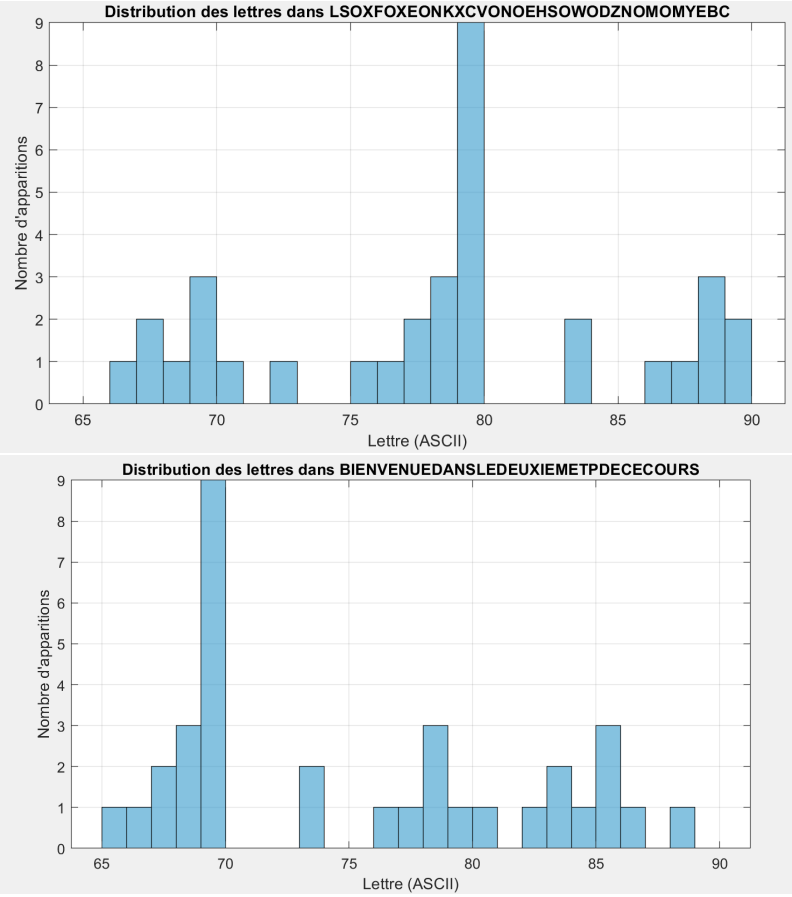

FIGURE 4 – La distribution d'apparition des lettres pour le mot codé et le message.

Remarque On voit bien que la distribution de l'un est la distribution de l'autre de manière décalé.

Remarque Une manière de casser le code serait de regarder la lettre qui apparaît le plus, et se dire que c'est un E (par exemple). En effet en français, le E apparait beaucoup plus souvent le Z par exemple.

2.2.2 La fonction `distrib` (8 points)

La fonction `distrib` calcule et affiche la distribution du message codé. Pour chaque lettre, vous devez affiché : "A - x" où x est le nombre de fois que A apparaît.

La fonction `distrib` a accès à la variable globale `encPhi` et `msg_size`. Vous n'avez pas besoin de garder en mémoire la distribution, l'affichage dans le terminal suffira.

Bloc de code 9 – Le sous-programme `distrib`.

```
;==========================================================
; Sous-programme : distrib
; Affiche la distribution dans le terminal
; Entree  : decPhi
; Sortie  : Sortie dans le terminal
;==========================================================

FORCE:    RET0
```

2.2.3 La fonction `FREQUENC` (8 points)

La fonction `FREQUENC` implémente l'attaque par analyse fréquentielle. Dans la fonction vous devez :

1. Faire une boucle qui demande une clé à essayer (Les clés à utiliser doivent être choisi en se basant sur le résultat de `distrib`)
2. Afficher le message décodé avec la clé proposée
3. Après chaque essai, le programme demande si la boucle doit continuer ou non

Bloc de code 10 – Le sous-programme `FREQUENC`.

```
;==========================================================
; Sous-programme : FREQUENC
; Attaque par analyse fréquenciel
; Entree  : decPhi
; Sortie  : Doit afficher les différents décodages, ainsi que la clé testée
;==========================================================

FORCE:    RET0
```

2.2.4 Complexité algorithmique

La complexité algorithmique peut être défini comme le nombre d'instructions réalisé pour faire votre programme. Une

Question (2 points) Quelle attaque est la plus complexe ?

Question (2 points) Combien d'instructions est nécessaire pour l'attaque par force brute ?

Question (2 points) Si la clé est trouvée du premier coup, combien d'instructions a été nécessaires avec l'analyse fréquentielle ?

2.2.5 Robustesse de votre code (10 points)

Ajouter deux directives `msg1` et `code1`, puis mettez à jour votre main, en ajoutant toutes les méthodes précédemment appliqué à `msg`, pour appliquer le codage de César aussi sur le nouveau message `msg1`.